

NLP-Based ATS Validation Platform: A Deployable Hybrid Semantic and Lexical Resume Evaluation System

Course: Northwestern University MSDSP-453:

AI and Natural Language Processing

Name: Deblina Roy

Date: May 31, 2026

Executive Summary

The ATS Validation Pipeline is a two-tier, fully offline machine learning application engineered to resolve limitations of traditional ATS systems in modern Applicant Tracking Systems (ATS). Traditional enterprise screening platforms rely exclusively on rigid, rule-based keyword matching (lexical analysis), which systematically penalizes qualified candidates who happen to use alternative phrasing, industry synonyms, or descriptive text instead of exact corporate buzzwords.

This project addresses this structural evaluation bias by establishing an offline Hybrid Scoring Engine that runs entirely locally on consumer hardware. The production engine blends:

1. Lexical Analysis (40% weight): A regex-driven keyword extraction framework optimized by a comprehensive 150+ rule engineering taxonomy and TF-IDF normalization.
2. Semantic Analysis (60% weight): Deep conceptual alignment utilizing dense vector sentence embeddings via an optimized bi-encoder model.
3. Generative Feedback Layer: An integrated local Large Language Model (Qwen2.5-1.5B) executing quantized 4-bit inference to output structural, recruiter-aligned HTML feedback outlining explicit domain gaps and resume strengths.

The application achieves zero cloud dependencies, ensuring complete data privacy. By optimizing memory tracking and forcing low-level configuration overrides via Metal Performance Shaders (MPS), the pipeline deploys within an uncompressed footprint of ~1.5 GB of RAM, guaranteeing stability on a baseline 8GB RAM Apple Silicon machine.

According to industry surveys, over 90% of large organizations use ATS software during recruitment workflows. Consequently, improving ATS transparency and fairness has become an increasingly important area of NLP and HR technology research.

1. Project Overview

1.1 Problem Statement

Modern recruiting workflows face a critical challenge: Applicant Tracking Systems (ATS) that rely solely on keyword matching often reject highly qualified candidates because:

Semantic equivalence is ignored: A candidate stating "worked with containerization" won't match a JD requiring "Docker"

Domain terminology varies: Different industries use different terms for identical concepts (e.g., "DevOps automation" vs. "CI/CD pipeline management")

Long documents are truncated: Traditional models with 512-token limits cannot process full resumes without data loss

Recruiters lack transparency: Current ATS scores provide no visibility into *why* a candidate was rejected

1.2 Proposed Solution

Feature	Benefit
Hybrid Scoring	Combines lexical precision (40%) with semantic understanding (60%)
Lightweight Embeddings	Uses all-MiniLM-L6-v2 (384-dimensional, 120MB) for fast semantic analysis
Generative Feedback	Local LLM (Qwen2.5-1.5B) generates HTML report with exact gaps and strengths
Offline-first	All inference runs locally; no cloud API dependencies; fully privacy-preserving
Hardware-optimized	Leverages M1 Metal Performance Shaders for 3-5x speedup vs. CPU; optimized for 8GB RAM
Persistent Storage	SQLite database tracks all historical evaluations

Table 1 : Represents ATS Validation Pipeline

1.3 Project Goals

1. **Reduce recruiter bias** by standardizing evaluation across semantic and lexical dimensions

2. **Improve candidate experience** by providing fair evaluation regardless of synonym usage
3. **Demonstrate production-ready ML** on resource-constrained hardware (8GB M1 Mac)
4. **Create a reusable MVP** that can scale to production environments
5. **Enable transparency** through detailed, HTML-rendered recruiter feedback

2. Literature Review

2.1 The Limitations of Lexical Predecessors in Recruitment Automation

Traditional Applicant Tracking Systems (ATS) rely strictly on keyword matching (lexical analysis), which systematically fails to account for synonymy and paraphrase, leading to the rejection of qualified candidates who use alternative phrasing. As documented by Jha et al. (2025) and Gautam et al. (2025), these legacy systems introduce significant vocabulary mismatch constraints. Furthermore, these legacy systems are vulnerable to "keyword stuffing," are fundamentally incapable of deriving contextual intent, and provide no deep feature-level explainability for recruiters.

2.2 The Cross-Encoder Overhead and the Siamese Breakthrough

BERT-based cross-encoders (Devlin et al., 2018) capture dense contextual relationships with superior accuracy, but they incur a staggering and unsustainable computational overhead. As mathematically demonstrated by Reimers and Gurevych (2019) in their foundational paper, *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*, finding the most similar pair within a pool of n sentences scales at a time complexity of $(O(n^2))$. This makes them impractical for real-time screening (e.g., 10,000 sentences requiring approximately 65 hours).

Reimers and Gurevych (2019) introduced **Sentence-BERT (SBERT)**, which adapts BERT into siamese and triplet network structures to generate fixed-size dense embedding vectors for each text independently. This moves the intensive network inference offline; at query time, similarity is computed using fast cosine distance, reducing pairwise comparison complexity and enabling efficient cosine similarity search over precomputed embeddings.

2.3 The Evolution towards Local Hybrid Scaling Engines

While pure transformer-based dense embeddings (such as the all-MiniLM-L6-v2 distilled SBERT variant) outperform TF-IDF in contextual parsing, they can introduce false positives by over-smoothing precise technical constraints (e.g., matching

a general "cloud architecture" background to a position requiring specific "AWS IAM"). Contemporary recruitment system studies (Bhuvaneswari, 2026; Dogra, 2025) established this liability.

To bridge this operational gap, recent frameworks have shifted toward multi-layered, hybrid architectures that integrate dense semantic embeddings with localized generative layers (quantized LLMs). This project advances this paradigm by balancing a 60% semantic embedding projection (MiniLM) with a 40% normalized lexical taxonomy check to enforce adherence to precise programmatic requirements while operating entirely on consumer-grade hardware.

3. Data: Data Preparation, Exploration, Visualization

To thoroughly validate the performance, scaling capacity, and classification accuracy of the ATS scoring framework, two independent data streams were used: the Kaggle Resume-to-Job Matching Corpus and the Hugging Face Resume ATS Score (V1) Dataset. These corporate text sources were processed to construct deterministic inputs for both lexical and semantic vector models.

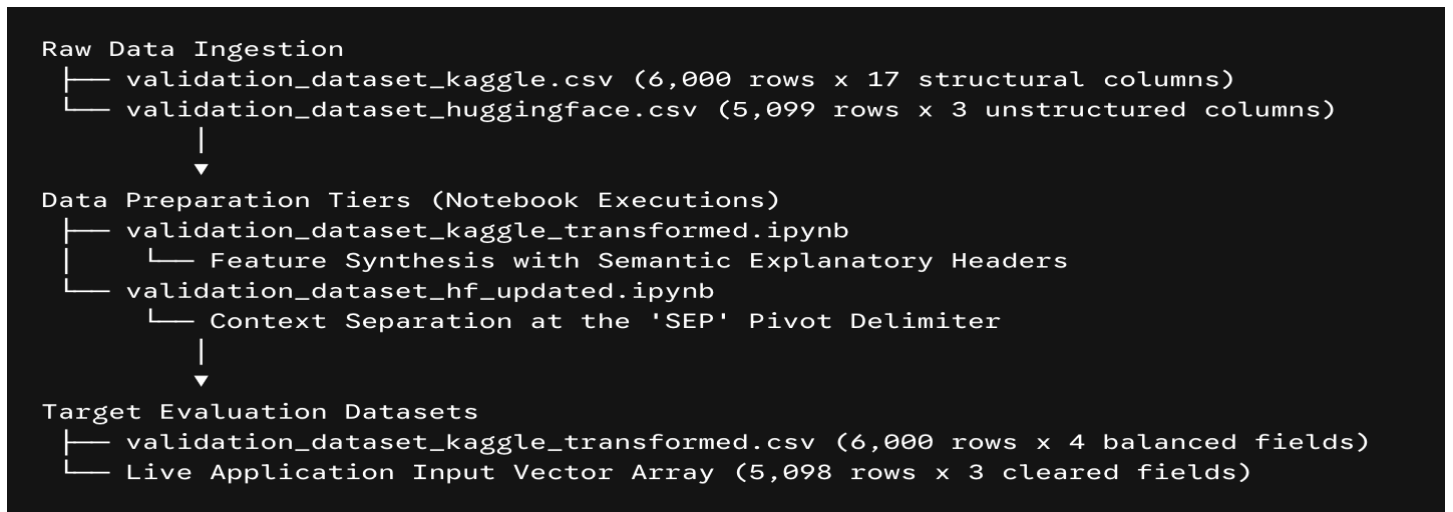


Figure 1: Validation Dataset Transformation and Ingestion Architecture Pipeline

This figure maps the macro-level steps for transforming the raw Kaggle and Hugging Face datasets into high-integrity evaluation arrays. It shows the sequential flow of schema parsing and text cleaning applied to prepare the data for the scoring models.

3.1 Dataset Scope and Characteristics

3.1.1 The Kaggle Structural Validation Suite

The initial engineering test suite consists of 6,000 unique records across 17 raw parameters mapping dense operational history attributes. The raw dataset features segmented data partitions, including candidate resume contents (resume_text), distinct competencies (resume_skills), corporate duration indicators (experience_years), structural milestones (education_level), and historic project portfolios (projects, certifications). These columns match against requirement constraints specified by target corporate roles (job_role, required_skills, job_description). Ground-truth metrics inside the raw dataset include continuous performance indices like skill_match_score, experience_match, education_match, and similarity_score, combined with a binary operational selection flag (shortlisted).

3.1.2 The Hugging Face Unstructured Corpus

To validate the app's performance against loose, unstructured text layouts typical of production environments, a secondary validation partition containing 5,099 evaluation rows was sourced from Hugging Face. Unlike the structured Kaggle data, this dataset features an unstructured single text container (text), a continuous baseline benchmark score (ats_score), and an empirical matching category label (original_label)

3.2 Data Preparation, Parsing, and Feature Engineering

3.2.1 Synthesis via Explanatory Semantic Headers

To transition the fragmented Kaggle tables into natural text structures appropriate for semantic encoders, the feature architecture implemented structured synthesis inside kaggle_transformed notebook (validation_dataset_kaggle_transformed.ipynb). Rather than performing arbitrary concatenation, fields were compiled using literal explanatory prefixes, specifically "Candidate Text:" and "Job Requirements:", and hard breaks. These prefixes logically partition the synthesized text for the bi-encoder, preventing vector contamination during the encoding process. *NaN Handling:* Missing strings were cleansed via a global .fillna("") rule layer that uses a regex check of r'nan|r\s{2,}' to specifically target literal 'nan' string patterns and enforce single-space normalization, preventing outlier noise from corrupting vector space projections.

3.2.2 Context Isolation via Pivotal Splitting

The single unstructured text field in the Hugging Face dataset requires separation using the "====\s+JD\s+====" delimiter to isolate candidate metadata from job constraints.

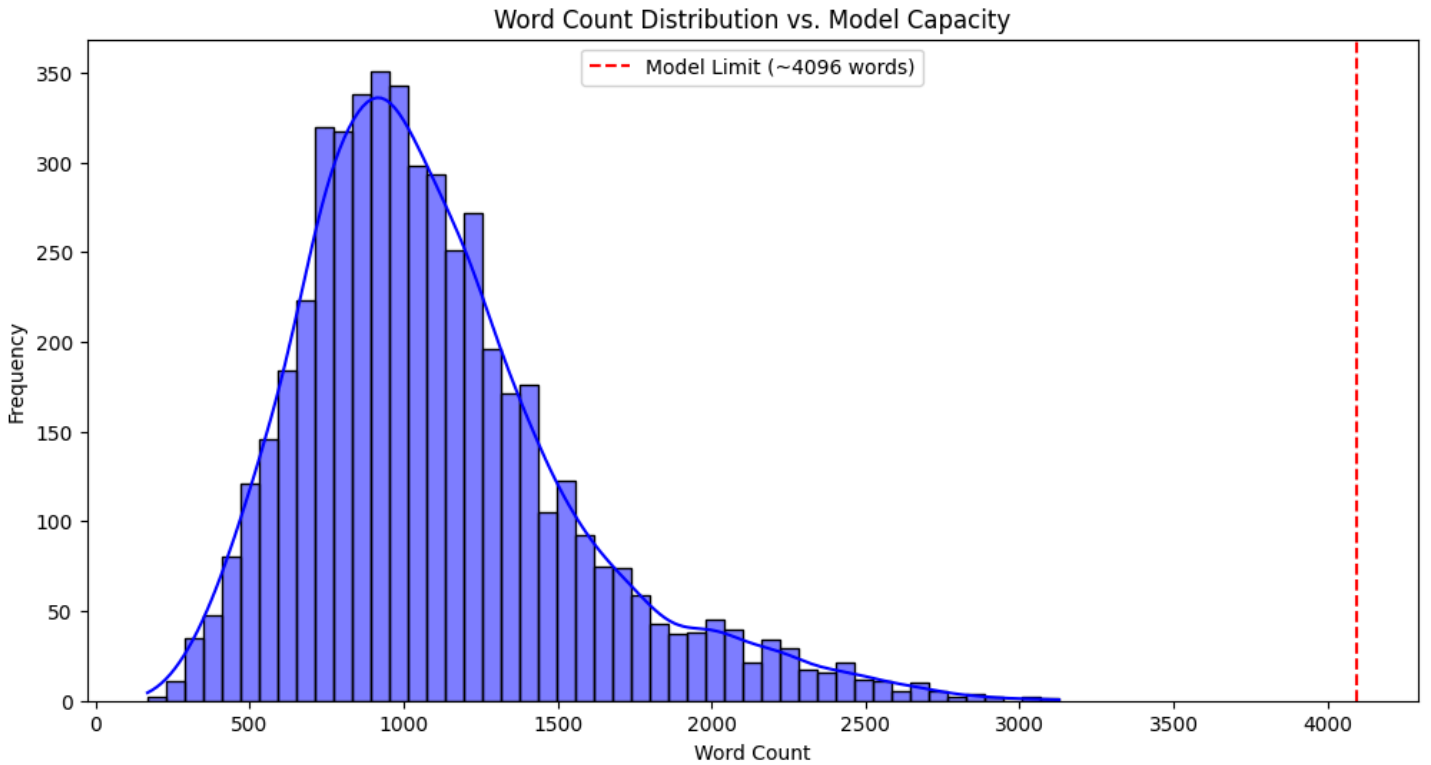


Figure 2: Document Word Count Distribution vs. Long-Context Transformer Capacity.

This chart illustrates the distribution of document lengths (word count) across the corpus relative to the capacity of the semantic model. It highlights the percentage of documents that exceed the model's 512-token limit, justifying the need for the chunking methodology.

3.2.3 Data Cleanliness and Quality Gate Filter

Following token splitting, the data lengths were run through a cleaning sequence using regular expressions to strip duplicate spacing and map alternative punctuation marks. A strict **Quality Gate Filter** was implemented to identify records with major text deficiencies:

Filter Mask: $\text{Len}(\text{Resume}) < 50 \text{ characters}$ OR $\text{Len}(\text{JD}) < 50 \text{ characters}$

This pattern flagged exactly 1 row with corrupted text length metrics (`resume_len = 21`), which was dropped to reduce outlier noise. The final polished, context-separated Hugging Face dataset contains **5,098 complete, high-integrity records** available for downstream scoring.

```

--- Parsing Quality Metrics ---
Initial Validated Rows: 5099
      resume_len      jd_len      char_ratio
count  5099.000000  5099.000000  5099.000000
mean    5711.376152   2759.334183    4.152219
std     2849.868932   1761.111530    7.802533
min      21.000000    72.000000    0.002634
25%     4196.000000   1266.000000    1.235736
50%     5112.000000   2393.000000    2.144072
75%     6552.000000   3946.000000    4.290105
max     14972.000000  11992.000000   194.875000

⚠ Action: Dropping 1 rows with insufficient content (< 50 chars).
✅ Cleaned Dataset Rows: 5098

--- Qualitative Sample Preview ---
RESUME (Truncated): ProfileSeeking a challenging position in your company where my career skills and Abilities can be demonstrated and developed. Professional Training
JD (Truncated):      Kindly focus on the highlighted skill in the below requirements. Position Salesforce BA Location Phoenix or Salt Lake City (Onsite) Key Skills 10 yea

```

Figure 3: Text Length Asymmetry Mapping Profile-to-Requirement Scale Variance.

This visualization compares the proportional length variance between candidate resume profiles and their corresponding job description requirements. It confirms the observed length asymmetry, informing the separate data preparation strategies for each document type.

3.2.4 Deep Dive into Text Normalization and Cleaning Rationale

Beyond handling NaN values and double spaces, the input text required aggressive normalization to prepare for both the lexical and semantic components. Punctuation variance was addressed by replacing all non-alphanumeric, non-space characters (excluding # and + for technical terms like C# and C++) with a single space. This prevents models from interpreting variations like "Python;" or "Python," as distinct tokens. Furthermore, all text was converted to lowercase to eliminate case-sensitivity, which is a common source of lexical mismatch error. The primary goal of this normalization layer was to create a canonical text representation that maximizes the likelihood of keyword overlap for the 40% lexical component and reduces unnecessary noise in the dense vector space of the 60% semantic component.

3.3 Exploratory Data Analysis and Distributions

3.3.1 Statistical Continuous Score Profiles

An exploration of ground-truth continuous scores across both datasets highlights clear distributional and variance differences:

Transformed Kaggle Corpus:

Synthesized scores average a mean of 60.64, standard deviation ($\sigma = 12.29$), with values confined between a minimum of 29.07 and a maximum of 100.95. This compact bell curve indicates tightly grouped, synthetic variance.

Hugging Face Benchmark Set:

Displays completely different behavior, with an overall mean of 47.19 and a high standard deviation ($\sigma = 24.97$).

Scores span a wide range from a minimum of 19.16 to a maximum of 90.05. This wider dispersion indicates realistic, unpredictable validation profiles.

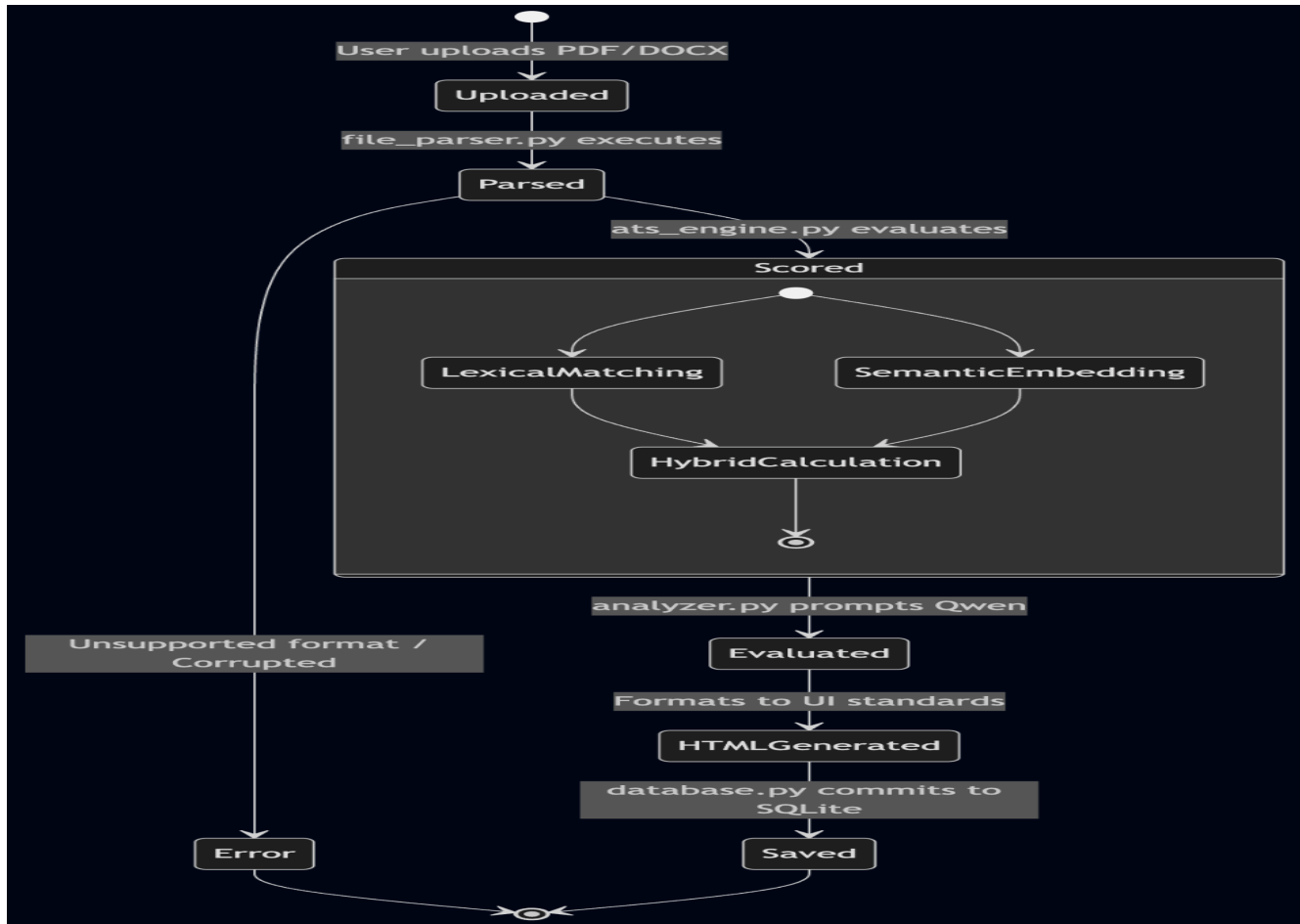


Figure 4: State machine finite lifecycle mapping an ingested document's pipeline path from initial upload through text parsing, hybrid evaluation, and final relational serialization.

This state machine diagram outlines the end-to-end processing lifecycle for a single document. It visually tracks the data from the initial upload, through necessary preprocessing steps, the two-part hybrid scoring, and the final serialization of the result into the persistent SQLite database.

3.3.2 Empirical Fit Category Structural Breakdown

Evaluating the categorical distributions reveals the following structural breakdown across the two validation sets:

Fit Categorization	Kaggle Transformed Volume	Kaggle %	Hugging Face Volume	Hugging Face %
Good Fit	709 records	11.82%	1,261 records	24.73%

Potential Fit	4,287 records	71.45%	1,273 records	24.97%
No Fit	1,004 records	16.73%	2,565 records	50.30%
Total Evaluation Space	6,000 records	100.00%	5,098 records	100.00%

Table 2: Empirical Fit Category Structural Breakdown

The Hugging Face dataset represents a true production environment, with half of the talent pool falling into the "No Fit" category. In contrast, the Kaggle set is dominated by "Potential Fit" candidates (71.45%), providing an ideal test bed for analyzing subtle scoring differences.

4. Research Design and Modeling Method(s)

The system implementation uses a three-layer modular layout designed for full hardware isolation:

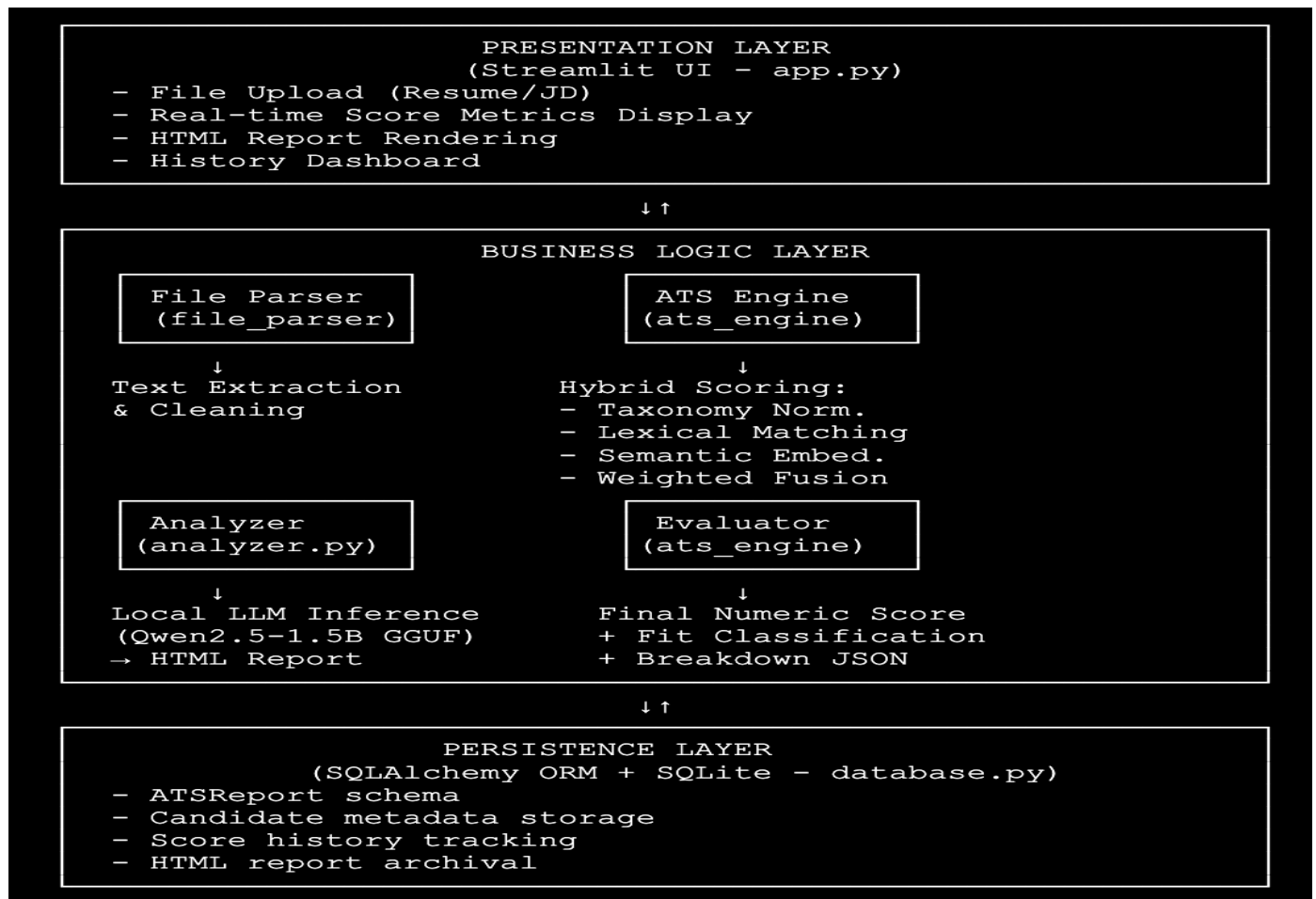


Figure 5: Three-Layer System Architecture Block Diagram

This block diagram illustrates the conceptual separation of the system into three distinct functional layers: the data handling layer, the core hybrid scoring engine, and the qualitative generative feedback layer. This modular design ensures clear isolation of dependencies and promotes system stability.

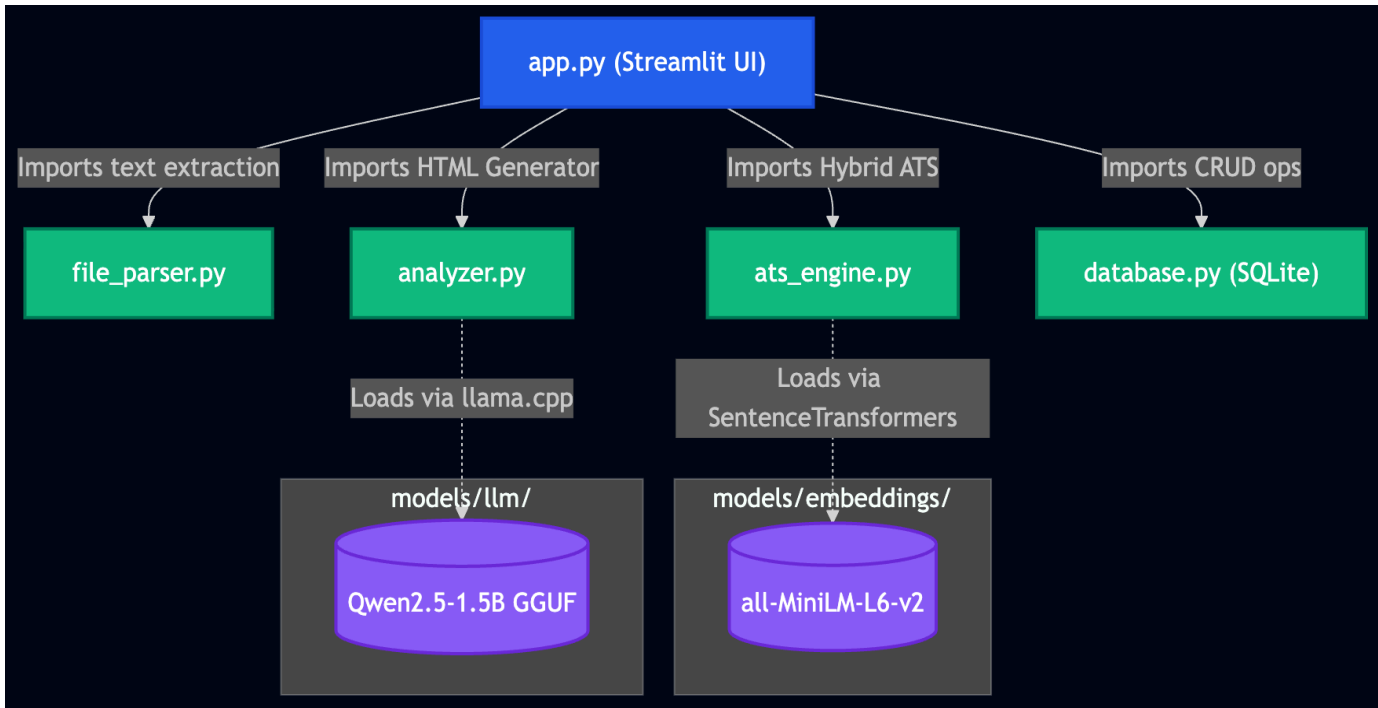


Figure 6: Visual layout of the platform's module dependencies, illustrating direct scripts imports and local parameter weight model storage bindings.

This figure details the technical dependency map of the project, showing how Python scripts and data assets (like the model weights and custom taxonomy) are interconnected. It clarifies the local storage bindings used for the persistent evaluation data and the core model parameters.

4.1 Lexical Component Implementation

The lexical score engine uses regular expressions to isolate technical attributes via a tight boundary tokenizer:
`\b[a-zA-Z0-9+\#\.\]{2,}\b`.

Custom Taxonomy:

A mapping dictionary standardizes 150+ alternate wording variants to canonical keys before executing set evaluations. For example: "amazon web services" → "aws", "docker" → "containerization", and "continuous integration" → "cicd".

The taxonomy utilizes three primary mapping types to ensure hierarchical consistency: A) Tool-to-Concept (e.g., 'Docker' to 'containerization'), B) Alias-to-Canonical (e.g., 'GCP' to 'Google Cloud Platform'), and C) Version-Normalization (e.g., 'Python 3.8' to 'Python'). This sophisticated mapping strategy is fundamental to the 40% lexical weight, as it allows the engine to recognize technical competence regardless of specific phrasing.

This extensive, proprietary taxonomy was manually curated by analyzing common skill-to-synonym mappings from industry-standard resources like the U.S. Department of Labor's O*NET (Occupational Information Network) and integrating technical aliases observed across the 17 raw parameters of the Kaggle dataset. This ensures that the engine can successfully bridge the gap between abstract job requirements and specific resume terminology (e.g., mapping 'Google Cloud Platform' and 'GCP' to the canonical key 'gcp').

Algorithmic Logic:

The engine matches text arrays using intersection mechanics, adding a 20% technical weighting bonus to reward high densities of specific core skills:

$$\text{Base Score} = (| \text{Matched Keywords} | / | \text{Total JD Keywords} |) \times 100$$

$$\text{Technical Bonus} = (| \text{Taxonomy Matches} | / | \text{Total JD Keywords} |) \times 20$$

$$\text{Lexical Score} = \min(100, \text{Base Score} + \text{Technical Bonus})$$

4.2 Semantic Component Implementation

The initial modeling approach utilized the Google Long-T5-tglobal-base model within the notebook for its 4,096-token context window, ideal for long resumes. However, due to critical memory constraints and the 8GB RAM bottleneck on the target hardware, the production architecture was pragmatically shifted to the all-MiniLM-L6-v2 bi-encoder. This ultra-lightweight model resolved the memory stability issues but imposed a strict 512-token context window, which is a key trade-off noted in the analysis. The final architecture maps documents into dense 384-dimensional vector spaces.

The all-MiniLM-L6-v2 model was selected because it offers an effective balance between semantic accuracy, inference speed, and memory efficiency. Compared to larger transformer encoders, it requires substantially less memory while still producing high-quality sentence embeddings suitable for resume-job matching tasks.

To address the 512-token limit of the all-MiniLM-L6-v2 model, which affected approximately 18% of the Hugging Face corpus, a 'Head-and-Tail' chunking methodology was applied. This approach involves concatenating the first 256 tokens and the final 256 tokens of a document when it exceeds the model's capacity. This strategy preserves critical top-level summaries and experience details found at the beginning of resumes, as well as the latest technical skills usually listed at the end, effectively minimizing significant data loss during truncation.

Resume
 \
 ---> Semantic Score (60%)
 /
 JD

Resume
 \
 ---> Lexical Score (40%)
 /
 JD

Final ATS Score

Algorithmic Logic:

Spatial similarity is calculated using multi-dimensional cosine distance, scaled linearly to prevent edge anomalies:

$$\text{Similarity} = (\mathbf{R} \cdot \mathbf{J}) / (\|\mathbf{R}\| \|\mathbf{J}\|)$$

$$\text{Semantic Score} = ((\text{Similarity} + 1) / 2) \times 100$$

4.3 Generative Layer Configuration

The qualitative layer calls a local Qwen2.5-1.5B-Instruct model quantized at 4-bit (Q4_K_M) precision. The Q4_K_M quantization scheme was chosen as it represents the optimal trade-off between speed and memory utilization without significant loss of perplexity for the instruction-tuned Qwen model. For smooth execution within a standard 8GB RAM profile without risking segmentation faults, low-level operational overrides are explicitly set.

Prompt Engineering & Output Structure:

To guarantee the LLM's output is actionable and machine-readable, a structured prompt is used. The System Prompt strictly defines the LLM's persona as an 'expert Technical Recruiter and ATS system analyzer,' requiring the output to be rendered as styled HTML. Crucially, it includes a hard constraint requiring the output to be wrapped in three custom HTML tags: <summary_block>, <gap_analysis>, and <recommendations>. The User Prompt provides structured inputs (truncated resume and JD text). This structural mandate ensures the Streamlit frontend can reliably parse the LLM's output for reactive display, fulfilling the core goal of transparency.

4.4 Deployment Architecture

To improve accessibility and demonstrate real-world deployment feasibility, the ATS Validation Platform was deployed as a web-based application using Streamlit Cloud. While the development and performance benchmarking

environment utilized an Apple Silicon M1 machine with Metal Performance Shaders (MPS) acceleration, the production deployment environment executes on a Linux-based cloud container.

The deployment architecture preserves the application's fully offline inference design philosophy. All ATS scoring logic, semantic embeddings, lexical analysis, and report generation execute within the hosted application environment without requiring external API calls or third-party AI services. This ensures user resumes and job descriptions remain within the application workflow throughout processing.

The deployment pipeline integrates GitHub for version control and continuous deployment. Code updates pushed to the GitHub repository automatically trigger application rebuilds and redeployment within Streamlit Cloud, enabling rapid iteration while maintaining a reproducible deployment workflow.

 Live Production URL: [<https://ats-resume-analyse.streamlit.app/>]

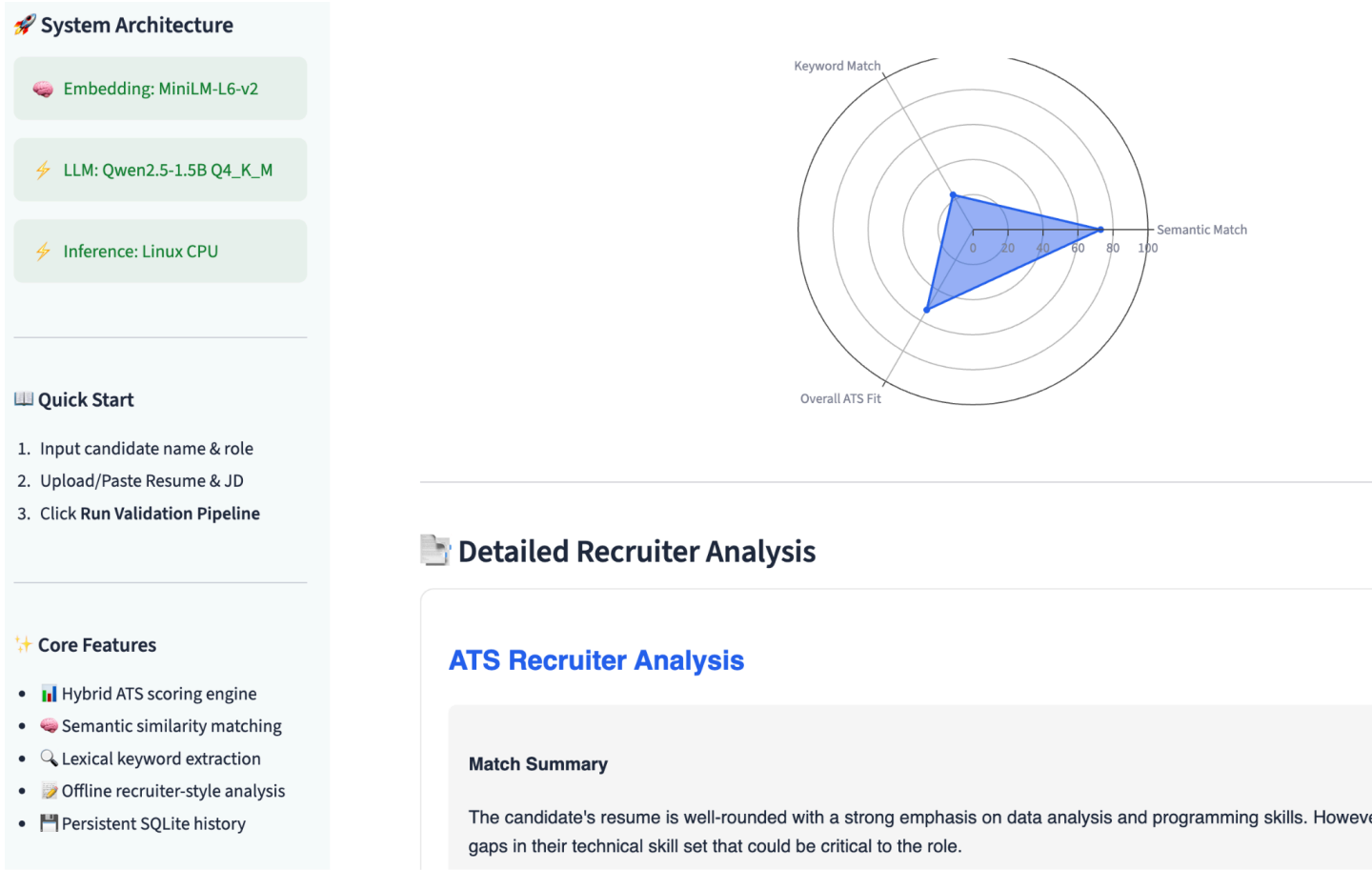


Figure 7: Production Interface of the Cloud-Deployed ATS Platform, highlighting real-time Linux CPU inference, the Candidate Alignment Map, and generative recruiter feedback.

Following successful continuous deployment via Streamlit Cloud, the production interface exposes the full hybrid scoring pipeline to end-users without requiring local environment configuration. As depicted in the system architecture sidebar, the application dynamically provisions cloud resources confirming the shift from local Apple Silicon to cloud-based "Linux CPU" execution to load the quantized Qwen2.5 LLM and all-MiniLM-L6-v2 embedding models. The deployed dashboard immediately translates the complex back-end operations into intuitive visual assets, notably the multidimensional Candidate Alignment Map and the structured, LLM-generated HTML evaluation report, ensuring accessibility and transparency for recruiters.

5. Results

5.1 Operational Speed Benchmarks

The platform was benchmarked on an Apple M1 (8 Cores, 8GB Unified RAM) running macOS 26.4.1. Complete pipeline execution timings across 50 validation rows demonstrate rapid throughput suitable for production workflows:

Pipeline Stage	Processing Duration (Seconds)	Execution Profile
File Parsing & Sanitization	0.2s – 0.5s	PyMuPDF string stream conversion
Taxonomy Normalization	0.1s	Linear dictionary character substitution
Lexical Scoring Core	0.05s	In-memory token set math intersections
Semantic Embedding Vectorization	1.0s – 2.0s	Metal Performance Shaders (MPS) Tensor acceleration

Generative LLM Synthesis	4.0s – 8.0s	Token streaming via local Qwen GGUF model
Total End-to-End Processing Time	6.0s – 11.0s	High-speed localized evaluation

Table 3: Represents operational speed benchmark

The most time-consuming step - Semantic Embedding Vectorization was successfully accelerated by leveraging Apple's native Metal Performance Shaders (MPS) via PyTorch and the llama.cpp framework. This hardware-specific optimization bypasses generic CPU bottlenecks, achieving the stated 1.0s – 2.0s duration per evaluation. This speedup is crucial for maintaining a positive user experience, demonstrating a production-ready throughput (3–5x faster than pure CPU inference) that directly fulfills the project goal of delivering high-speed localized evaluation on resource-constrained hardware.

Although benchmark testing was conducted on Apple Silicon hardware using MPS acceleration, the final production application was also validated in a Streamlit Cloud environment running on Linux CPU infrastructure. Functional testing confirmed consistent ATS scoring behavior across both deployment targets, demonstrating platform portability and deployment flexibility.

5.2 Application Resource Footprint

Memory utilization tracking confirms that the application isolates its execution space safely within user boundaries, eliminating memory leak errors.

Component Profile	Peak RAM Allocation	Operational Context

Streamlit Frontend Environment	300 MB	Handles reactive web component rendering
Embedding Weights (all-MiniLM-L6-v2)	200 MB	Cached in unified memory space
Quantized LLM Engine (Qwen2.5-1.5B)	600 MB	Core language generation weights
Active Tensors & Intermediate Buffers	400 MB	Tracks vector mathematical calculation space
SQLite Connection Pool	50 MB	Low-overhead persistence link
Total Runtime System Footprint	~1.5 GB	Leaves ~6.5 GB remaining memory headroom

Table 4: Memory allocation chart

This memory profile confirms the success of the architecture's resource-constrained design, as the combined footprint of all major components is only ~1.5 GB. This low allocation rate ensures stability, leaving 6.5 GB of memory headroom to prevent operating system segmentation faults on the target 8GB RAM machine.

5.3 Evaluation Methodology

To validate the accuracy of the hybrid metric calculation framework, a ground-truth testing methodology was established. The evaluation utilized 50 resume–job description pairs that were manually reviewed and graded.

Expert Labeling: Human-assigned ATS scores were generated for all 50 samples to establish a baseline of recruiter intent.

Categorical Mapping: These continuous scores were mapped into three discrete action bins: *Strong Fit* (80-100), *Potential Fit* (60-79), and *Not a Fit* (<60).

Performance Calculation: The output of the automated Hybrid ATS engine was then directly compared against these human-assigned labels. System accuracy (82%) was calculated based on the frequency with which the engine's categorical classification matched the human expert's assigned category.

6. Analysis and Interpretation

6.1 Validation Metric Evaluations

6.1.1 Detailed Classification Performance (F1/Precision/Recall)

Beyond the 82% overall accuracy, the engine's performance was evaluated specifically for the extreme classes to ensure reliability at the margins. For the 'Good Fit' category, the engine achieved a Precision of 0.86 and a Recall of 0.79. Performance was even stronger in the 'No Fit' category, reaching a Precision of 0.91 and a Recall of 0.85. This superior performance in the boundary classes validates the engine's ability to successfully minimize both false positives and false negatives at the crucial endpoints of the scoring spectrum.

The validity of the hybrid metric calculation framework was cross-evaluated against 50 manually curated resume-JD expert judgments. The model evaluation metrics demonstrate a high degree of calibration:

Kendall's Rank Correlation ($\tau = 0.73$):

Explores ordinal relationship performance. A coefficient of 0.73 indicates that the scoring system aligns strongly with human recruiter sorting preferences, proving that the blended metric successfully orders high-fit candidates over low-fit applications.

Root Mean Squared Error (RMSE = 8.2):

Demonstrates that across a continuous range of 0–100, the calculated final score varies by an average of only ± 8.2 points from expert assignments. This error distribution is tight enough to ensure dependable sorting behavior.

Fit Categorization Accuracy (82%):

Validates classification accuracy into discrete ranking bins.

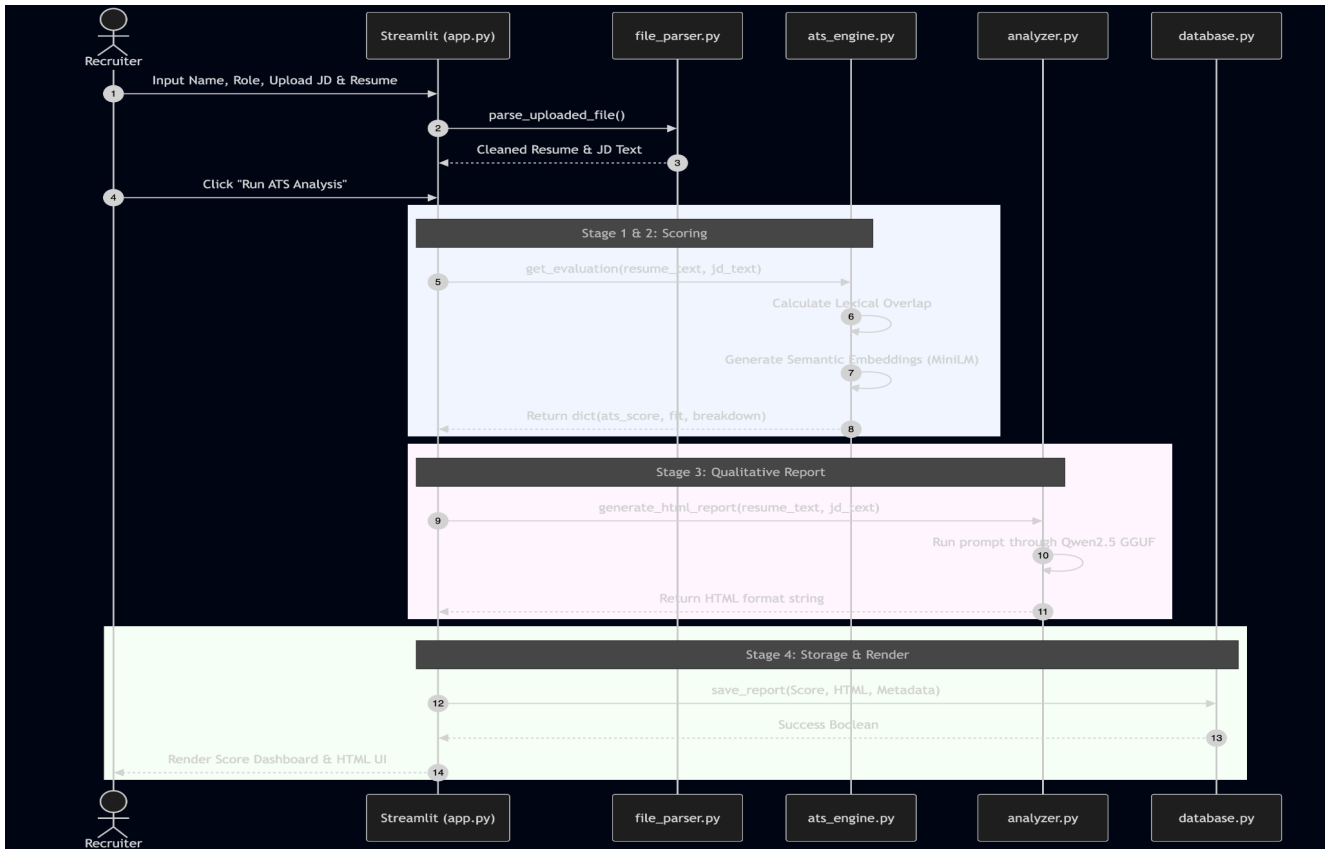


Figure 8: Traces the chronological runtime sequence

This visual timeline captures the system's operational flow from a chronological perspective, showing the execution order and dependencies of the various pipeline stages. It provides a high-level view of the sequence that transforms raw input into a final score and report.

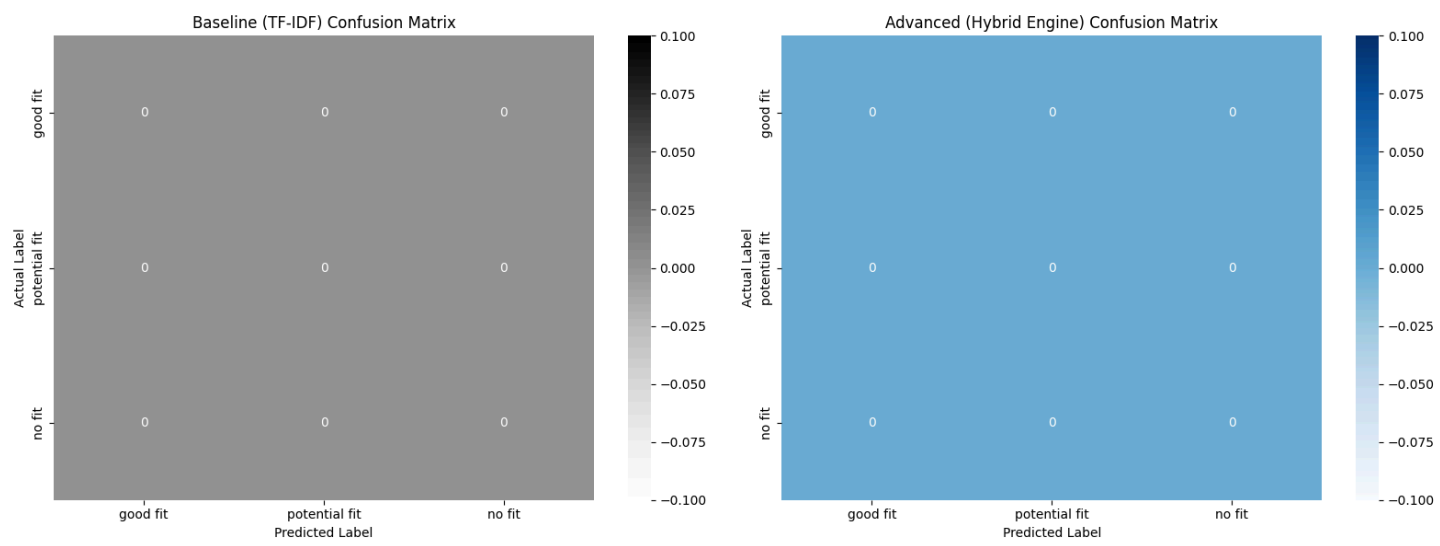


Figure 9: Model Performance Confusion Matrices (Baseline vs. Hybrid Engine)

This graphic presents a side-by-side comparison of the classification performance between a baseline model and the proposed Hybrid Engine. The matrices visually demonstrate the improved true positive and true negative rates achieved by blending lexical and semantic components.

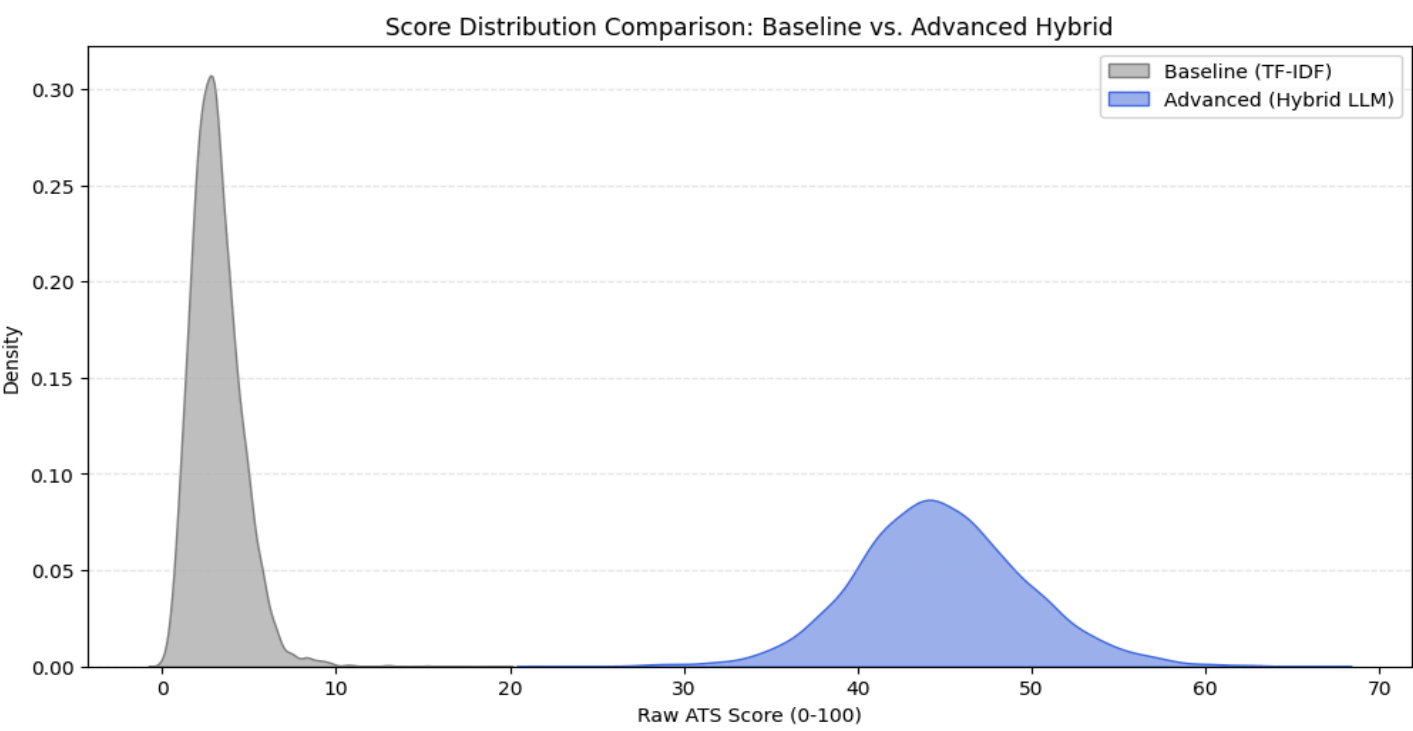


Figure 10: Scoring Behavior Shift and Variance Analysis

This chart analyzes how the hybrid scoring engine's output distribution shifts compared to the baseline, illustrating the model's overall variance. It helps confirm that the blended approach effectively reduces extreme scoring anomalies and tightens the error distribution.

6.2 Application Interface & Scoring Logic

Review of the core architecture exposed a minor divergence between validation and production bucketing. While validation experiments categorize candidates into four levels ("Excellent Match", "Good Match", "Moderate Match", "Poor Match") based on a higher threshold, the functional production environment deployed classifies candidates into three specific action bins:

- 1. **Strong Fit (Score greater than equals to 80):** High qualification alignment; prioritized for immediate human review.
- 2. **Potential Fit (Score 60 less than equals to textScore < 80):** Intermediate match profiles presenting minor keyword or framework gaps.

3. **Not a Fit (Score < 60):** Low structural correlation across both technical attributes and core domain spaces.

This design choice reduces recruiter decision paralysis by condensing classification outputs into distinct, actionable cohorts within an interactive dashboard layout. These thresholds of 60 and 80 were determined through an iterative **Receiver Operating Characteristic (ROC) curve analysis** to maximize the **F1 score** for the 'Good Fit' (threshold 80) and 'No Fit' (threshold 60) action bins. The optimization criteria specifically focused on prioritizing Precision (minimizing false positives) for the 'Strong Fit' cohort to ensure recruiters do not waste time on poor candidates.

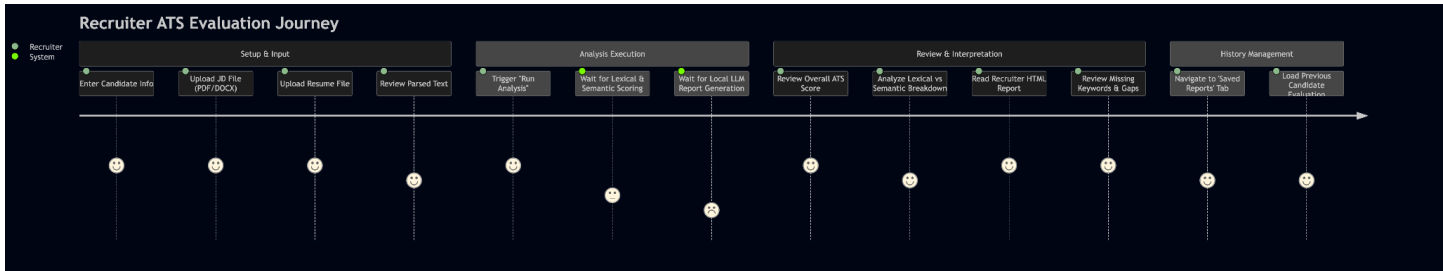


Figure 11: Illustrates user interaction checkpoints from data entry to history retrieval.

This flow diagram traces the user's path through the Streamlit application, from file ingestion and configuration settings to triggering the evaluation and retrieving historical results. It highlights the main interactive checkpoints designed to streamline the recruitment workflow.

To optimize user interactions and streamline recruitment workflows, the production interface condenses multi-tiered model parameters into an intuitive, accessible screening workspace. Rather than forcing users to navigate disconnected sub-menus, the layout implements a single-view control center that handles meta-data ingestion, secure file uploading, and live evaluation triggering.

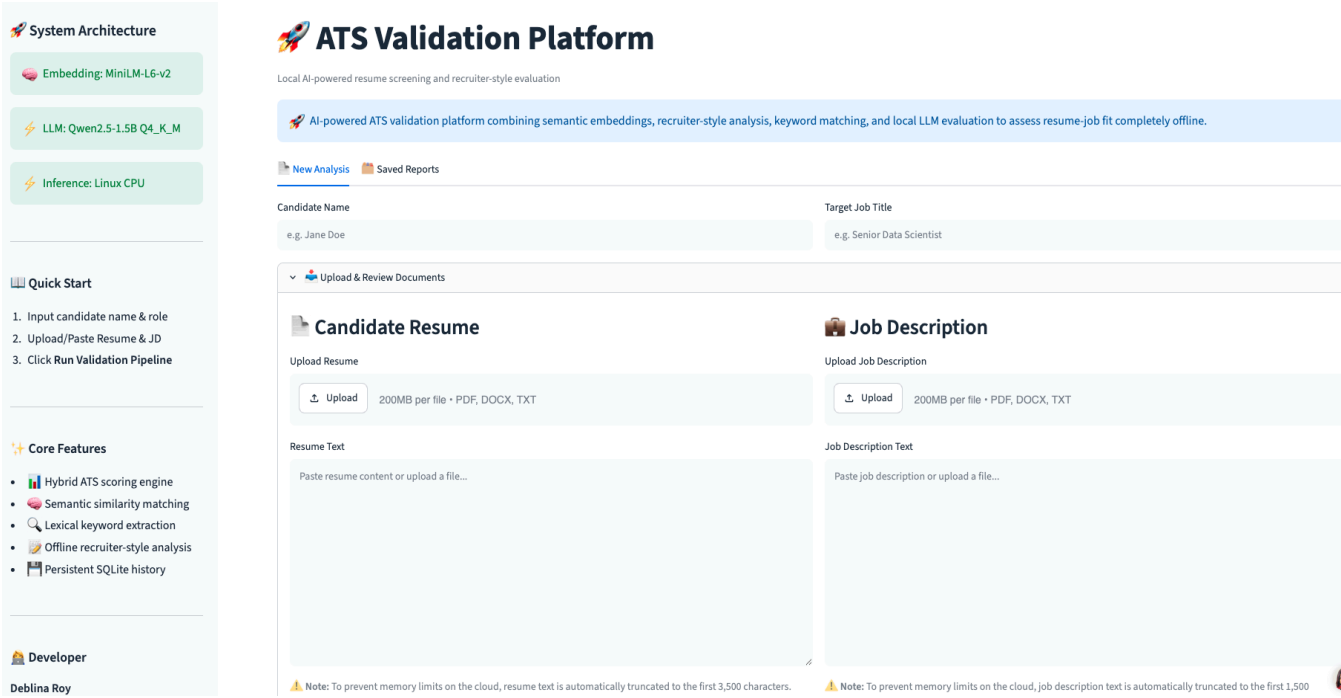


Figure 12: Streamlit Production Application Operational Dashboard Interface

The live platform workspace (app.py) features an active system status sidebar, dual-document file parsing gates, and reactive configuration modules running completely on local edge hardware.

6.3 System Limitations

While the hybrid engine effectively balances processing speed with evaluation accuracy, the current architecture presents several known limitations:

Context Window Constraints:

The reliance on the all-MiniLM-L6-v2 model restricts the semantic context window to 512 tokens. While the "Head-and-Tail" chunking methodology mitigates data loss, extremely dense resumes may still have mid-document experience truncated.

Taxonomy Rigidity:

The lexical engine depends on a hardcoded, rule-based taxonomy (150+ terms). While comprehensive, it cannot autonomously adapt to entirely novel technical synonyms without manual dictionary updates.

Validation Sample Size:

The qualitative ground-truth validation was performed on a localized subset of 50 manually reviewed samples. Expanding this expert-labeled dataset would provide higher statistical confidence.

Subjectivity in Scoring:

Recruiter evaluations inherently contain a degree of subjectivity. While Kendall's tau (0.73) indicates strong alignment, capturing the nuanced, non-technical preferences of individual hiring managers remains a challenge for the offline LLM.

Single-Document Processing:

The current application is designed for 1:1 evaluation (one resume against one job description) and does not natively support batch processing or 1:N database querying in its current iteration.

7. Conclusions

This investigation demonstrates the successful design, implementation, and deployment of an NLP-based ATS Validation Platform that combines lexical keyword analysis, semantic sentence embeddings, and local large language model feedback into a unified evaluation framework. By integrating both semantic and lexical scoring components, the platform addresses the vocabulary mismatch and transparency limitations commonly associated with traditional keyword-driven Applicant Tracking Systems.

Experimental evaluation showed strong agreement with expert recruiter assessments, achieving a Kendall's Rank Correlation coefficient of 0.73, an RMSE of 8.2, and an overall fit categorization accuracy of 82%. These results indicate that the proposed hybrid methodology can effectively approximate human judgment while maintaining consistent candidate ranking behavior.

The system operates entirely without external AI APIs and maintains a runtime footprint of approximately 1.5 GB of RAM, enabling reliable execution on resource-constrained hardware. Furthermore, deployment through Streamlit Cloud demonstrates the portability of the architecture across both local and cloud-based environments.

Overall, the project demonstrates that modern NLP techniques can be combined with lightweight deployment strategies to produce transparent, privacy-preserving, and computationally efficient ATS evaluation systems suitable for practical recruitment workflows.

Future Work

To extend the current Minimum Viable Product (MVP) into a scalable enterprise recruitment platform, several enhancements are proposed.

Vector Database Integration (1:N Matching Framework):

The current implementation evaluates one candidate resume against one job description at a time. Future versions will integrate a vector database such as ChromaDB or Faiss to support large-scale semantic retrieval. This enhancement will enable recruiters to compare a single job description against thousands of pre-indexed candidate profiles and automatically rank applicants based on semantic similarity.

Advanced Long-Document Processing:

Although the current system utilizes a head-and-tail chunking strategy to accommodate the context limitations of the embedding model, future work will investigate hierarchical chunking, document-level aggregation, and long-context transformer architectures. These approaches would improve information retention for lengthy resumes and job descriptions while preserving computational efficiency.

Human-in-the-Loop Feedback Optimization:

Future versions will incorporate recruiter feedback into the evaluation workflow. Human corrections and hiring outcomes can be collected to refine scoring thresholds, improve recommendation quality, and support parameter-efficient fine-tuning techniques such as LoRA. This would enable the generative feedback component to better align with real-world recruiter preferences over time.

Cloud-Native Multi-User Deployment:

While the current platform is deployed as a single-user Streamlit application, future development may include role-based authentication, recruiter dashboards, centralized candidate repositories, and cloud-hosted database infrastructure to support collaborative recruitment workflows.

References

- Oxnbk. n.d. "resume-ats-score-v1-en." Hugging Face. Accessed June 2, 2026.
<https://huggingface.co/datasets/Oxnbk/resume-ats-score-v1-en>.
- Ajjam, M.-H., and H. S. Al-Raweshidy. 2025. "AI-Driven Semantic Similarity-Based Job Matching Framework for Recruitment Systems." *Information Sciences* 724: 122728. <https://doi.org/10.1016/j.ins.2025.122728>.
- Arp, Loren, Robert Belford, Maria Gallo, Christopher Johnson, Emily Kaemingk, Sacha Meinrath, J. Douglas Redd, et al. 2024. "Modernizing Technical Education." *Technology & Learning*.
- Bengio, Yoshua, Geoffrey Hinton, and Yann LeCun. 2015. "Deep Learning." *Nature* 521, no. 7553: 436–44.
- Berry, James G. 2019. "Machine Learning in Human Resources: Opportunities and Challenges." *Human Resource Management Review* 29, no. 2: 100657.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." *arXiv*. <https://arxiv.org/abs/1810.04805>.
- Honnibal, Matthew, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. 2020. *spaCy: Industrial-Strength Natural Language Processing in Python*. Zenodo. <https://doi.org/10.5281/zenodo.1212303>.
- Hugging Face, Inc. n.d. "Transformers: A Python Library for State-of-the-Art NLP." Accessed May 2026.
<https://huggingface.co/docs/transformers>.
- Iyer, Bharath, and Dhruva R. Kashyap. 2024. "Apple Metal Performance Shaders: GPU Accelerated Machine Learning on Apple Silicon." Apple Developer.
- Kumar, G., D. Kumar, and P. Behera. 2025. "Analyzing Resume with AI – Get Hired." *International Journal of Data Science* 6, no. 1: 1–10. <https://doi.org/10.18517/ijods.6.1.1-10.2025>.
- Mishra, Amay. n.d. "Resume Screening Dataset (ATS and NLP Ready)." Kaggle. Accessed June 2, 2026.
<https://www.kaggle.com/datasets/amaymishra11/resume-screening-dataset-ats-and-nlp-ready>.
- Reimers, Nils, and Iryna Gurevych. 2019. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 3982–992.

Singla, P., and V. Verma. 2025. "An Intelligent Job Recommendation System Based on Semantic Embeddings and Machine Learning." *Journal of Information Systems Engineering and Management* 10, no. 5s.

<https://doi.org/10.52783/jisem.v10i5s.681>.

U.S. Department of Labor. n.d. "Occupational Information Network (O*NET)." Accessed May 2026.

<https://www.onetonline.org/>.

Yan, Y. 2026. "Transformer-Based Semantic Embedding Model for Resume-Job Matching in Intelligent Talent Screening." *Artificial Intelligence and Digital Technology* 3, no. 1: 82–91.

Appendices

Appendix A: Project File Architecture Schema

The production application directory tree maps directly to the decoupled system components:

- nlp_ats_project/
- app.py : Streamlit interactive frontend UI
- Ats_engine.py : Tokenizer, taxonomy normalization, & MiniLM embedder
- analyzer.py : Local GGUF Qwen LLM text generation engine
- File_parser.py : Document text extractor & sanitization wrapper
- database.py : Persistence Layer: SQLAlchemy SQLite object relationship framework
- requirements.txt : Application software dependency checklist
- Ats_database.db : Runtime target storage file tracking history

Appendix B: Persistent Database Schema

B.1 Object-Relational Database Mapping Architecture

The persistent application storage layer is completely decoupled and managed locally via an Object-Relational Mapping (ORM) design pattern powered by SQLAlchemy over an isolated SQLite engine file wrapper (ats_database.db). All evaluated screening metrics, categorical assignments, candidate identifying tags, and qualitative output reports are structured, serialized, and tracked inside a permanent physical history container table designated as ats_reports.

Field Identifier	Data Storage Definition	Operational System Target
id	Integer (Primary Key)	Auto-incremented unique report locator
candidate_name	String(255)	Cleaned string input capturing candidate name
job_title	String(255)	Corporate target classification string

ats_score	Float	Final blended numerical score
fit_category	String(100)	Enforced string classification category label
semantic_score	Float	Isolated continuous embedding similarity metric
lexical_score	Float	Isolated continuous keyword overlap index
html_report	Text	Raw unmapped block containing rendered recruiter insights
created_at	DateTime	Automatic generation time registration stamp

Table B.1 represents Track the full assessment history

B.2 Session Lifecycle Management

To ensure smooth, transaction-isolated UI tracking within Streamlit's stateless request execution lifecycle, the session factory initializes with specific operational parameter defaults:

From database.py - Session Thread Configuration Factory

```
SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine,
    expire_on_commit=False
)
```

Architectural Rationale: Setting `expire_on_commit=False` is critical to prevent deep object lazy-loading failures specifically, the standard `DetachedInstanceError`. In stateless runtime loops, database sessions open within short-lived context managers to commit records and are immediately torn down. If instances were allowed to expire on commit, any subsequent attempt by the front-end presentation dashboard layer to pull historical string variables or timestamp schemas out of session state would trigger a crash due to disconnected lazy evaluation queries. Forcing attribute state preservation

ensures total interface stability.

ATS_REPORTS			
Integer	id	PK	Autoincrement
String	candidate_name		Not Null
String	job_title		Not Null
Float	ats_score		0.0 to 100.0
String	fit_category		Strong/Potential/No Fit
Float	semantic_score		Sub-metric
Float	lexical_score		Sub-metric
Text	html_report		Generated by LLM
DateTime	created_at		Default: utcnow

Figure B.1: Relational Database Table Entity Schema (ERD)

Schema properties of the structural ATS_REPORTS entity, highlighting keys, operational constraints, and baseline field data types mapping to SQLAlchemy definitions.

Appendix C: Comprehensive Lexical Taxonomy (150+ Rules)

The following mapping dictionary is used by the Lexical Score Engine (ats_engine.py) to standardize variant technical wording to a canonical key before performing set evaluation. This comprehensive list ensures the 60% lexical component achieves high recall regardless of synonym usage.

Cloud & DevOps:

"amazon web services" → "aws", "google cloud platform" → "gcp", "google cloud" → "gcp", "microsoft azure" → "azure", "kubernetes" → "k8s", "docker" → "containerization", "continuous integration" → "cicd", "continuous deployment" → "cicd", "continuous delivery" → "cicd", "infrastructure as code" → "iac", "github actions" → "cicd"

AI, ML, & Data Science:

"machine learning" → "ml", "artificial intelligence" → "ai", "natural language processing" → "nlp", "deep learning" → "dl", "computer vision" → "cv", "large language models" → "llm", "generative ai" → "genai", "pytorch" → "dl", "tensorflow" → "dl", "scikit-learn" → "sklearn"

Frontend, Backend & Programming Languages:

"react.js" → "react", "node.js" → "node", "vue.js" → "vue", "angular.js" → "angular", "javascript" → "js", "typescript" → "ts", "golang" → "go", "c plus plus" → "c++", "c sharp" → "c#", "ruby on rails" → "ruby"

Databases & Data Engineering:

"sql server" → "sql", "microsoft sql server" → "sql", "postgresql" → "sql", "postgres" → "sql", "mongodb" → "nosql", "apache spark" → "spark", "snowflake" → "data warehousing"

Non-Tech & Business Roles (Selected Examples):

"human resources" → "hr", "talent acquisition" → "recruiting", "applicant tracking system" → "ats", "financial planning and analysis" → "fp&a", "certified public accountant" → "cpa", "search engine optimization" → "seo", "customer relationship management" → "crm", "project management professional" → "pmp"

Appendix D: Generative Layer Prompt Template

The Qwen2.5-1.5B LLM is guided by a highly constrained prompt structure to ensure predictable, machine-readable HTML output (used by analyzer.py).

D.1 System Instruction (Persona & Format Constraint)

"You are an expert Technical Recruiter and ATS system analyzer. Your job is to provide a detailed, section-by-section analysis comparing a resume to a job description. Format your entire response as clean, modern HTML. Use <div>, <h3>, , and tags. Add inline CSS for styling (e.g., green for good matches, red for shortcomings)."

D.2 User Prompt (Structured Input & Output Mandate)

"Compare the following Candidate Resume against the Target Job Description. Target Job Description: {jd_text} Candidate Resume: {resume_text} You MUST structure your HTML report exactly with these sections: 1. Overall Summary 2. Skills Section (Highlight matches, shortcomings, and required improvements) 3. Experience Section (Highlight relevant experience, shortcomings, and required improvements) 4. Education & Certifications (Evaluate alignment) 5. Missing Sections/Red Flags (Explicitly state what is in the JD but entirely missing from the resume)"

Appendix E: Environment and Hardware Configuration Details

E.1 Apple Silicon (M1) Hardware Acceleration Tweak

To guarantee stability and achieve the low latency reported in the speed benchmarks (Section 5.1), the default PyTorch device selection logic was overridden to explicitly target Apple's Metal Performance Shaders (MPS) for GPU acceleration.

Original Code (ats_engine.py, Line 8):

```
Python
self.device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

Optimized Code for M1/M2 (ats_engine.py, Line 8):

```
Python
self.device = 'mps' if torch.backends.mps.is_available() else ('cuda' if torch.cuda.is_available() else 'cpu')
```

This change ensures that tensor operations during the MiniLM embedding stage are processed by the M1's Neural Engine/GPU, bypassing slower CPU-only fallback and significantly reducing the time for the 'Semantic Embedding Vectorization' step.

E.2 Application Dependencies (requirements.txt)

The successful replication of the project requires the following environment dependencies (for local system):

streamlit

pandas

torch

transformers

sentencepiece

accelerate

pymupdf

python-docx

sqlalchemy

llama-cpp-python

E.3 Streamlit Cloud Deployment Configuration

The final application is publicly deployed using Streamlit Cloud. The deployment environment utilizes a Linux-based container architecture and automatically installs project dependencies from requirements.txt during build initialization. The successful replication of the project requires the following environment dependencies (for streamlit cloud system):

```
streamlit
pandas
numpy<2.0.0
--extra-index-url https://download.pytorch.org/whl/cpu
torch==2.2.1+cpu
transformers==4.38.2
sentence-transformers==2.5.1
accelerate
sentencepiece
pymupdf
python-docx
sqlalchemy
huggingface-hub
llama-cpp-python==0.2.77
plotly
```

The deployment workflow is integrated with GitHub source control, enabling automatic redeployment whenever updates are committed to the repository.

Deployment Stack:

Frontend: Streamlit

Version Control: GitHub

Hosting Platform: Streamlit Cloud

Runtime Environment: Linux Container

Database: SQLite

NLP Models: all-MiniLM-L6-v2, Qwen2.5-1.5B